

Using try-except Blocks

When you think an error may occur, you can write a try-except block to handle the exception that might be raised. You tell Python to try running some code, and you tell it what to do if the code results in a particular kind of exception.

Here's what a try-except block for handling the `ZeroDivisionError` exception looks like:

```
try:
    print(5/0)
except ZeroDivisionError:
    print("You can't divide by zero!")
```

We put `print(5/0)`, the line that caused the error, inside a try block. If the code in a try block works, Python skips over the except block. If the code in the try block causes an error, Python looks for an except block whose error matches the one that was raised, and runs the code in that block.

In this example, the code in the try block produces a `ZeroDivisionError`, so Python looks for an except block telling it how to respond. Python then runs the code in that block, and the user sees a friendly error message instead of a traceback:

```
You can't divide by zero!
```

If more code followed the try-except block, the program would continue running because we told Python how to handle the error. Let's look at an example where catching an error can allow a program to continue running.

Using Exceptions to Prevent Crashes

Handling errors correctly is especially important when the program has more work to do after the error occurs. This happens often in programs that prompt users for input. If the program responds to invalid input appropriately, it can prompt for more valid input instead of crashing.

Let's create a simple calculator that does only division:

```
division
_calculator.py

print("Give me two numbers, and I'll divide them.")
print("Enter 'q' to quit.")

while True:
    ❶ first_number = input("\nFirst number: ")
        if first_number == 'q':
            break
    ❷ second_number = input("Second number: ")
        if second_number == 'q':
            break
    ❸ answer = int(first_number) / int(second_number)
        print(answer)
```
